Green DevOps – Energy Efficient CI/CD Pipeline

R26-SE-027

Project Proposal Report

M.P.M Thassara – IT22281432

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

March 2026

1. DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Thassara M.P.M.	IT22281432	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Mr.Jagath Wickramarathna	2026.03.11	

2. ABSTRACT

Modern cloud-native applications heavily rely on DevOps platforms such as Kubernetes to automate deployment, monitoring, and resource scaling in containerized environments. These technologies significantly improve system performance, scalability, and service availability. However, most DevOps infrastructures primarily focus on performance and reliability while overlooking energy efficiency and environmental sustainability. The rapid expansion of cloud computing has significantly increased the energy consumption of data centers, contributing to global carbon emissions and environmental impact [5], [11]. Studies show that cloud data centers consume a large portion of global electricity, and inefficient resource allocation can further increase carbon footprints in large-scale cloud environments [2], [5].

Traditional cloud auto-scaling mechanisms such as Kubernetes Horizontal Pod Autoscaler (HPA) operate using reactive threshold-based strategies, where scaling decisions are made only after CPU or memory utilization exceeds predefined limits. This reactive approach may lead to delayed responses, inefficient resource allocation, and excessive idle containers that consume unnecessary energy [4]. Furthermore, most DevOps platforms do not incorporate carbon-aware resource management or workload prioritization, resulting in inefficient scheduling and increased operational costs.

To address these limitations, this research proposes an AI-driven Green DevOps operational framework designed to improve cloud resource efficiency while minimizing environmental impact. The proposed system integrates three intelligent components: an AI Workload Prediction Engine, a Carbon Emission Optimization Engine, and a Job Priority Scheduling Engine. The AI prediction engine analyzes historical operational metrics—including CPU usage, memory utilization, network traffic, and request rates—to forecast future workload demand using machine learning techniques such as Long Short-Term Memory (LSTM) models [1]. These predictions enable proactive scaling decisions that allocate resources before workload spikes occur.

The carbon optimization engine evaluates potential scaling strategies by estimating energy consumption and corresponding carbon emissions, allowing the system to select resource allocation strategies that minimize environmental impact while maintaining performance [2], [12]. In addition, the job priority scheduling engine classifies workloads based on priority levels, ensuring that critical tasks such as authentication or payment processing are executed immediately, while low-priority background tasks can be delayed or consolidated to reduce energy consumption [3].

The proposed framework will be implemented using modern DevOps and cloud technologies including Kubernetes, Prometheus, Grafana, Docker, and machine learning frameworks. The system will be evaluated by comparing its performance with traditional reactive autoscaling mechanisms, using metrics such as workload prediction accuracy, resource utilization efficiency, system latency, energy consumption, and carbon emission reduction.

The expected outcome of this research is the development of a sustainable Green DevOps operational model that enhances cloud resource management while supporting environmentally responsible computing practices. By integrating predictive intelligence, carbon-aware optimization, and priority-based workload scheduling, the proposed framework contributes to the advancement of sustainable cloud infrastructure and green computing initiatives.

Keywords: Green DevOps, Predictive Auto-Scaling, Carbon-Aware Optimization, AI Workload Prediction, Sustainable Cloud Computing.

3. ACKNOWLEDGEMENT

I would like to sincerely thank my research supervisor, Mr. Jagath Wickramarathne, for his continuous support, valuable guidance, and constructive feedback throughout the preparation of this research proposal. His experience and advice greatly helped in shaping the direction and improving the quality of this study.

I am also grateful to my co-supervisor, Ms. Madusha Sulakshi Weerasooriya, for her encouragement, helpful recommendations, and support during the development of this research work. Her insights were very helpful in refining the ideas presented in this proposal.

I would also like to express my appreciation to our external supervisor, Mr. Dilanka Nayanajith, for providing valuable industry knowledge and practical perspectives that helped us better understand the real-world applications related to this research.

In addition, I would like to thank the academic staff of the Sri Lanka Institute of Information Technology (SLIIT) for the knowledge, guidance, and learning environment they provided, which supported the successful completion of this proposal.

Finally, I would like to acknowledge my team members Mendis J.D.L., Rathnayake D.M.B.H., and Kodithuwakku I.P. for their cooperation, discussions, and teamwork during the initial stages of this research project.

I also extend my gratitude to everyone who supported this work directly or indirectly during the preparation of this research proposal.

4. TABLE OF CONTENT

1. DECLARATION 2
2. ABSTRACT 3
3. ACKNOWLEDGEMENT 4
4. TABLE OF CONTENT 5
5. LIST OF TABLES 7
6. LIST OF FIGURES 8
7. LIST OF ABBREVIATIONS 9
8. LIST OF APPENDICES 10
9. INTRODUCTION 11
   9.1 Background and Context 11
   9.2 Literature Review 12
   9.3 Research Gap 12
10. OBJECTIVE 13
   10.1 Main Objective 13
   10.2 SMART Objectives 13
   10.3 Specific Objectives 13
11. METHODOLOGY 14
   11.1 Research Approach 14
   11.2 Technical Methods and Algorithms 14
   11.3 Tools and Platforms with Justification 15
   11.4 Data Collection and Ethical Considerations 16
   11.5 Validation Strategy and Performance Metrics 16
12. HIGH-LEVEL SYSTEM ARCHITECTURE 17
   12.1 Overview of the Proposed Solution and Its Components 17
   12.2 System Diagram of Components 17
   12.3 Individual Components and Their Parts 18
   12.4 Component Interactions 21
13. Data Flow Explanation 21
   12.5 Scalability and Security Considerations 23
14. USER REQUIREMENTS 24
   13.1 Stakeholder Identification 24
   13.2 Functional Requirements 25
   13.3 Non-Functional Requirements 25

15. COMMERCIALIZATION PLAN 26
14.1 Target Market and Customer Persona 26
14.2 Value Proposition 27
14.3 Revenue Model 27
14.4 Cost Estimation 28
14.5 Pricing Strategy 28
14.6 Competitive Advantage 29
14.7 Intellectual Property Considerations 29
16. BUDGET AND JUSTIFICATION 30
17. WORK BREAKDOWN STRUCTURE (WBS) 31
18. GANTT CHART 33
REFERENCES LIST 34
19. APPENDICES 35
19.1 Detailed Diagrams 35
19.2 Risk Analysis 37

5. LIST OF TABLES

Table 1: Tools and Platforms with Justification 15

Table 2: Cost Estimation 24

Table 3: Budget and Justification 27

6. LIST OF FIGURES

Figure 1: High-Level System Architecture 18
Figure 2: AI Autoscaling Model Architecture 19
Figure 3: AI Workload Prediction Engine Workflow 20
Figure 4: Persona 1 23
Figure 5: Persona 2 23

7. LIST OF ABBREVIATIONS

Abbreviation	Full Meaning
AI	Artificial Intelligence
CI/CD	Continuous Integration / Continuous Deployment
CO_2	Carbon Dioxide
CPU	Central Processing Unit
DSR	Design Science Research
HPA	Horizontal Pod Autoscaler
K8s	Kubernetes
LSTM	Long Short-Term Memory
OCI	Oracle Cloud Infrastructure
RAM	Random Access Memory
SSD	Solid State Drive
VPA	Vertical Pod Autoscaler

8. LIST OF APPENDICES

Appendix 1: High-Level System Architecture Diagram 23

Appendix 2: Carbon Optimization Decision Process 23

Appendix 3: Overview Diagram 23

9. INTRODUCTION

9.1 Background and Context

Cloud-native applications have transformed the way modern organizations develop, deploy, and manage software. Using microservices architecture, containerization, and automated DevOps pipelines, companies can achieve faster delivery cycles and scalable infrastructure [1]. Platforms such as Kubernetes enable container orchestration, automated scaling, and monitoring to maintain service performance and availability [2]. DevOps integrates development and operations processes to enable continuous integration, continuous deployment, and rapid delivery of applications [10]. With the adoption of containerization technologies and orchestration platforms such as Kubernetes, organizations can dynamically scale applications to meet user demand. However, the rapid growth of cloud computing has significantly increased the energy consumption of data centers, raising concerns about environmental sustainability and carbon emissions [8], [9].

Despite these advantages, most DevOps systems focus primarily on performance and uptime, often neglecting energy efficiency and environmental impact. Idle containers, over-provisioned pods, and unnecessary nodes consume electricity without performing meaningful work. Global studies indicate that data centers contribute nearly 1% of the world's electricity usage, and this percentage continues to grow with digital service expansion [3]. Such inefficiencies not only increase operational costs but also contribute significantly to carbon emissions, impacting the environment and climate change.

In real-world scenarios, organizations running large-scale cloud services—such as e-commerce platforms, financial systems, and data analytics—experience peaks and troughs in workload. Current reactive auto-scaling mechanisms respond only after CPU, memory, or network thresholds are exceeded. This can lead to delayed scaling, higher latency, and wasted energy [4]. Furthermore, cloud systems often treat all jobs equally, meaning low-priority batch jobs may consume resources while critical tasks like user authentication, payments, or real-time analytics face delays.

The proposed solution is an AI-driven Green DevOps framework, integrating three key components:

- AI Workload Prediction Engine: Uses historical system metrics to forecast future workloads, allowing proactive scaling.
- Carbon Emission Optimization Engine: Evaluates energy consumption and reduces carbon footprint by scaling only necessary resources.
- Job Priority Scheduling Engine: Ensures high-priority jobs run first while intelligently delaying low-priority tasks to save energy.

This approach aligns with the United Nations Sustainable Development Goal 12 (Responsible Consumption and Production), promoting environmentally sustainable cloud operations. It also falls under the research cluster of Green IT and Sustainable Cloud Computing, contributing to both academic research and practical implementation in enterprise cloud environments.

9.2 Literature Review

Recent studies highlight the need for energy-efficient and sustainable cloud operations:

- Predictive Scaling: Xu et al. [1] proposed using LSTM models for workload prediction to proactively scale resources, improving performance but without considering carbon emissions.
- Carbon-Aware Optimization: Li et al. [2] studied energy-efficient resource allocation in cloud data centers, showing that workload consolidation can reduce idle resources but lacked integration with real-time job prioritization.
- Job Priority Scheduling: Chen & Wang [3] explored priority-based cloud scheduling, allowing critical tasks to complete on time, but without combining AI prediction or carbon-aware decisions.
- Limitations of Reactive Scaling: Patel et al. [4] demonstrated that traditional Kubernetes HPA/VPA reacts only after thresholds are exceeded, causing latency and unnecessary energy consumption.

Existing research addresses individual components of predictive scaling, carbon optimization, or job prioritization but no single framework combines all three approaches to create a truly sustainable and efficient DevOps operational system.

9.3 Research Gap

From the literature, the following gaps are evident:

1. Lack of Carbon Awareness: Most systems ignore the environmental impact of scaling decisions, resulting in wasted energy and higher carbon emissions [2].
2. Reactive vs. Predictive: Existing DevOps systems react only after thresholds are exceeded, rather than predicting workload spikes in advance [1], [4].
3. No Integrated Job Prioritization: Cloud platforms do not combine workload prediction with intelligent prioritization, leading to inefficient resource usage [3].
4. Fragmented Solutions: AI workload prediction, carbon optimization, and job prioritization have been explored separately but not in an integrated system [5].

The proposed AI-driven Green DevOps framework addresses these gaps by combining predictive scaling, carbon-aware optimization, and job priority scheduling into a unified system. This approach ensures proactive scaling, reduced energy consumption, and efficient execution of critical tasks, providing both environmental and operational benefits. Some studies propose energy-efficient resource allocation strategies or introduce sustainability metrics for monitoring IT infrastructure [9], [11]. Other research highlights the potential of carbon-aware workload scheduling to reduce carbon emissions in cloud environments [12].

10.OBJECTIVE

10.1 Main Objective

The main objective of this research is to develop an AI-driven Green DevOps framework that can proactively scale cloud resources, optimize energy usage, and prioritize tasks. The aim is to ensure that cloud-native systems maintain high performance and low latency while minimizing idle resource consumption and reducing carbon emissions, contributing to more sustainable and efficient operations.

10.2 SMART Objectives

- **Specific:** Focus on combining predictive workload forecasting, carbon-aware resource optimization, and intelligent job scheduling in a single framework to tackle reactive scaling and energy inefficiency.
- **Measurable:** Success will be measured through quantitative improvements, such as reduced idle pod usage, lower system latency, and decreased energy consumption and CO_2 emissions.
- **Achievable:** The framework leverages existing cloud technologies (Kubernetes, Prometheus, Grafana) and AI models (LSTM for prediction), making it technically feasible.
- **Relevant:** The objectives directly address key challenges in cloud operations, including reactive scaling, inefficient resource utilization, and lack of sustainability.
- **Time-bound:** The research will complete design, implementation, and evaluation within the defined project timeline, demonstrating measurable improvements by the end of the study.

10.3 Specific Objectives

1. **Implement the Carbon Emission Optimization Engine** to monitor resources, estimate energy consumption and CO_2 emissions, and execute scaling decisions that reduce unnecessary energy use.
2. **Develop the AI Workload Prediction Engine** to forecast CPU, memory, network traffic, and request loads, enabling proactive scaling and reducing idle resource usage.
3. **Design the Job Priority Scheduling Engine** to ensure high-priority jobs (e.g., payments, authentication) run immediately while delaying low-priority tasks (e.g., backups, batch processing) to save energy.
4. **Integrate all three engines** into a single operational framework to provide a holistic solution for sustainable, efficient, and balanced cloud operations.
5. **Evaluate framework performance** using metrics such as CPU/memory utilization, active pod count, system latency, energy usage, and carbon emissions to demonstrate measurable improvement over traditional approaches.

11.METHODOLOGY

11.1 Research Approach

This research follows a Design Science Research (DSR) approach, which focuses on building and evaluating an innovative artifact—the AI-driven Green DevOps framework integrating predictive scaling, carbon optimization, and job prioritization [1]. The DSR methodology involves:

1. Identifying the core problem of reactive scaling, energy waste, and inefficient scheduling in cloud operations.
2. Establishing measurable objectives based on problem analysis.
3. Designing and developing the integrated framework.
4. Demonstrating and evaluating its effectiveness through experiments.

The DSR approach is suitable because it emphasizes solution construction and evaluation while contributing knowledge and reusable artifacts to the research community.

11.2 Technical Methods and Algorithms

1. AI Workload Prediction Engine:

- **Algorithms:** Long Short-Term Memory (LSTM) and sequence prediction algorithms are used to forecast workloads (CPU, memory, traffic, requests).
- **Method:** Historical operational metrics (e.g., CPU utilization, pod count) are used to train the model, which predicts near-future workloads to enable proactive scaling.
- **Justification:** LSTM models are effective at capturing long-term dependencies in time-series data, improving forecast accuracy [1].

2. Carbon Emission Optimization Engine:

- **Algorithms/Methods:** Energy estimation models calculate predicted energy usage and corresponding CO_2 emissions for potential scaling actions.
- **Method:**
    1. Monitor current resource usage and idle containers.
    2. Use established energy-to-carbon models to calculate expected emissions.
    3. Choose scaling decisions minimizing emissions while meeting demand.
- **Justification:** Carbon-aware optimization improves sustainability without sacrificing performance [2], [5].

3. Job Priority Scheduling Engine:

- **Algorithm:** Priority-aware scheduling algorithms, such as weighted fair scheduling and deadline-aware queueing, are used to manage task priorities.
- **Method:**

1. Classify workloads into priority levels (high, medium, low).
2. Schedule jobs based on priority, combining low-priority jobs when resource availability is limited.

- **Justification:** Priority scheduling ensures critical tasks are executed efficiently while conserving energy [3], [6].

4. Integration Strategy:

- The AI workload predictions and carbon optimization decisions jointly guide scaling and scheduling.
- Valuable metrics flow from Prometheus/Grafana into model inputs and scheduling decisions.

11.3 Tools and Platforms with Justification

Component	Tools/Platforms	Justification
AI Prediction	Python, TensorFlow	Widely used for building and training deep learning models [7], [6]. TensorFlow provides support for LSTM and time-series prediction essential for workload forecasting. Python offers extensive libraries for data preprocessing, model evaluation, and visualization, making it suitable for predictive scaling in cloud environments.
Monitoring & Metrics	Prometheus, Grafana	Prometheus collects metrics efficiently from Kubernetes clusters, while Grafana provides interactive dashboards for real-time monitoring. Together, they help administrators visualize CPU, memory, network usage, and pod activity, enabling proactive resource management and data-driven decision-making.
Container Orchestration	Kubernetes	Kubernetes provides automated deployment, load balancing, and scaling of containers. Its native support for Horizontal Pod Autoscaler (HPA) and custom metrics makes it ideal for integrating AI-driven predictive scaling and optimal pod management. It ensures high availability and reliability of cloud applications.
Simulation & Evaluation	Docker, Python (Pandas)	Docker allows isolated and reproducible simulations of cloud workloads, ensuring testing without affecting production. Python and Pandas help analyze metrics, simulate scheduling strategies, and evaluate performance, enabling efficient validation of AI models, carbon optimization, and job scheduling decisions.
Energy/Carbon Estimation	Standard Conversion Models	Energy-to-carbon conversion models calculate CO_2 emissions from predicted resource usage. These models are critical for carbon-aware scaling and scheduling, helping the

		system minimize energy consumption and environmental impact while maintaining performance [2], [8].

Table 1 : Tools and Platforms with Justification

11.4 Data Collection and Ethical Considerations

Data Collection:

- Metrics from Kubernetes clusters (CPU, memory, network traffic, pod counts).
- Historical usage logs from applications or open cloud datasets.
- Energy conversion data from established environmental datasets.

Ethical Considerations:

- No personal or sensitive user information will be collected.
- Production logs must be anonymized before use.
- All simulation and experimental data are handled according to standard ethical research practices, ensuring fairness and transparency.

11.5 Validation Strategy and Performance Metrics

Validation Strategy:

- Introduce diverse cloud workloads (steady, bursty, idle periods) to test system responses.
- Compare proposed framework with traditional reactive autoscaling (e.g., Kubernetes HPA/VPA).
- Analyze performance under varying priority levels and resource constraints.

Performance Metrics:

1. **Prediction Accuracy:** Percentage of correct workload forecasts (e.g., Mean Absolute Error, Root Mean Square Error) [1], [7].
2. **Resource Utilization:** Efficiency of CPU/memory usage and pod deployment.
3. **Scaling Responsiveness:** Reaction time to workload changes compared to reactive scaling.
4. **Energy Consumption:** Total energy consumed by active and idle resources.
5. **Carbon Emissions:** Estimated CO_2 emissions calculated from energy usage [2], [5].
6. **Latency and Throughput:** System response time and throughput for high-priority and normal tasks [3], [6].

12.HIGH-LEVEL SYSTEM ARCHITECTURE

12.1 Overview of the Proposed Solution and Its Components

The proposed system is an AI-driven Green DevOps framework designed to optimize cloud resource usage while reducing energy consumption and carbon emissions. The system integrates predictive analytics, carbon-aware resource optimization, and intelligent job scheduling within a Kubernetes-based cloud environment.

The architecture consists of several major components that work together to manage workloads efficiently:

1. Monitoring and Metrics Collection Layer
2. AI Workload Prediction Engine
3. Carbon Emission Optimization Engine
4. Job Priority Scheduling Engine
5. Container Orchestration Layer (Kubernetes)

These components interact continuously to analyze workload patterns, predict future demand, allocate resources intelligently, and maintain system performance while minimizing environmental impact.

12.2 System Diagram of Components

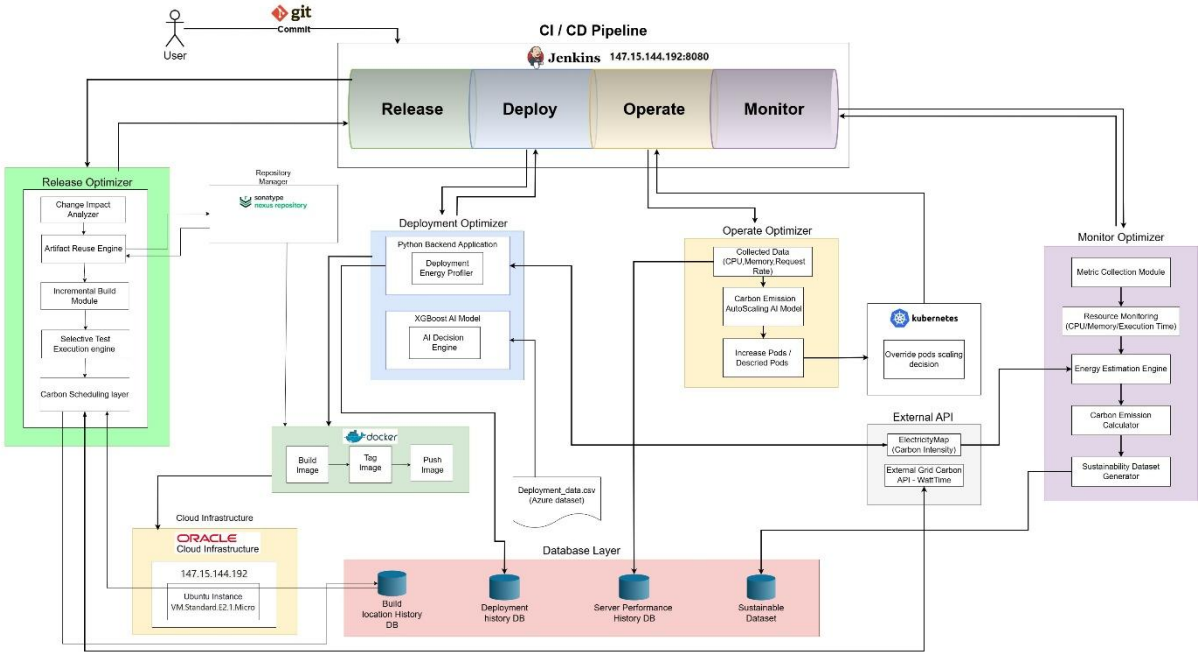


Figure 1: High-Level System Architecture

12.3 Individual Components and Their Parts

1. Monitoring and Metrics Collection Layer

This component collects real-time operational metrics from the Kubernetes environment.

Parts of this component include:

- **Prometheus** for collecting time-series metrics such as CPU usage, memory utilization, network traffic, and pod activity.
- **Grafana dashboards** for visualizing performance metrics and monitoring system health.

These metrics provide the raw data used by the AI prediction and optimization engines.

2. AI Workload Prediction Engine

This component analyzes historical and real-time metrics to forecast future system workloads.

Parts include:

- **Data preprocessing module** for cleaning and normalizing monitoring data.
- **LSTM-based prediction model** that detects patterns and predicts upcoming workload demand.
- **Prediction output module** that sends forecast results to the optimization engine.

This engine enables proactive scaling rather than reactive resource allocation.

3. Carbon Emission Optimization Engine

The carbon optimization engine evaluates scaling decisions based on both performance and environmental impact.

Its parts include:

- **Energy estimation module** that calculates expected energy usage of resources.
- **Carbon emission calculator** that converts energy consumption into CO_2 emission estimates.
- **Optimization decision module** that selects the most energy-efficient scaling option.

The goal is to reduce unnecessary resource usage while maintaining performance.

4. Job Priority Scheduling Engine

This component ensures that critical tasks receive priority in resource allocation.

It includes:

- Job classification module that categorizes tasks as high, medium, or low priority.
- Scheduling algorithm module that determines execution order.
- Job merging and delay logic for low-priority tasks to reduce energy usage.

This engine ensures critical services such as payments or authentication are always prioritized.

5. Container Orchestration Layer

The orchestration layer is responsible for deploying and managing application containers.

Key parts include:

- Kubernetes cluster nodes
- Pod management system
- Autoscaling mechanisms

The system dynamically scales pods based on recommendations from the AI and optimization engines.

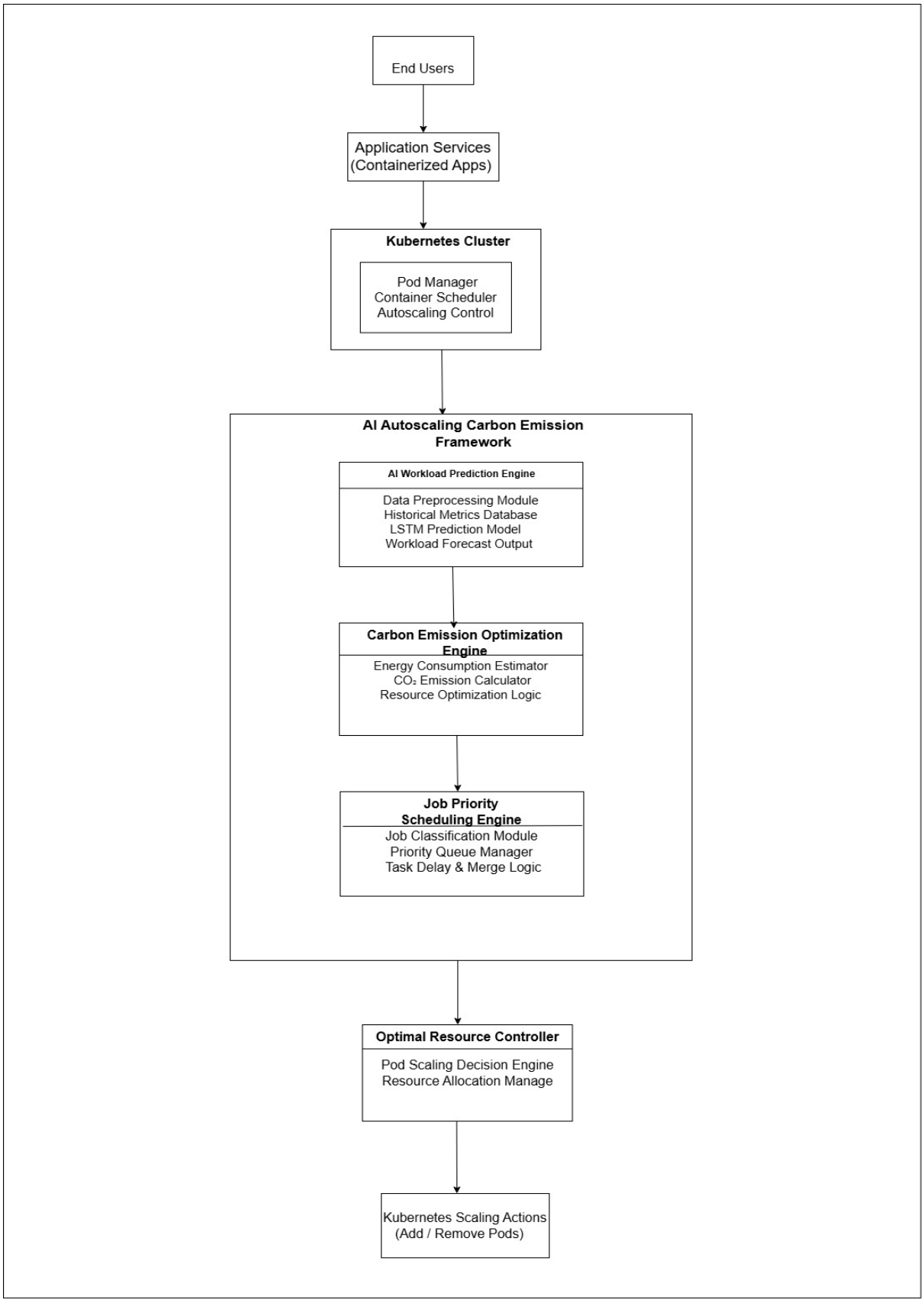


Figure 2: AI Autoscaling Model Architecture

12.4 Component Interactions

The components interact in a continuous feedback loop.

1. The monitoring layer collects metrics from the Kubernetes cluster.
2. The AI prediction engine analyzes historical and real-time data to forecast future workloads.
3. The carbon optimization engine evaluates predicted workloads and determines the most energy-efficient scaling strategy.
4. The job scheduling engine prioritizes tasks based on importance and system availability.
5. The orchestration layer executes scaling actions, deploying or terminating pods as required.

This interaction ensures that scaling decisions are proactive, optimized, and environmentally responsible.

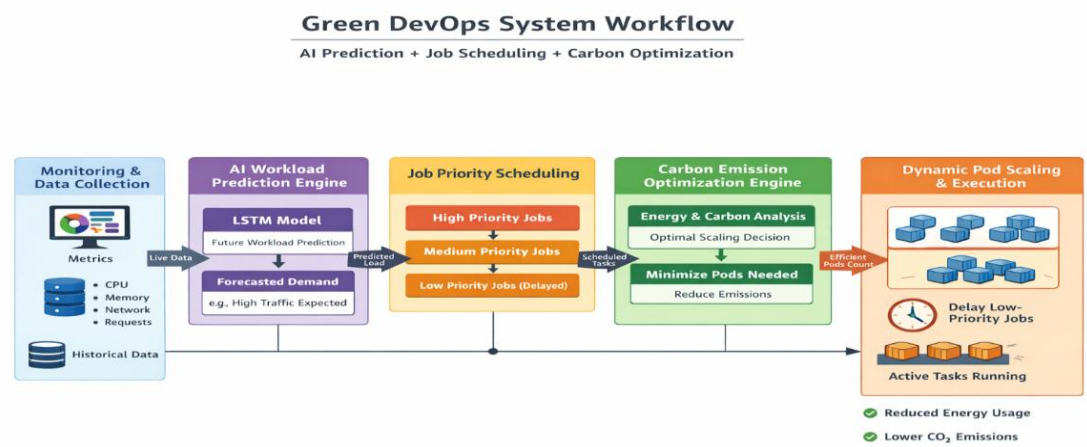


Figure 3: AI Workload Prediction Engine Workflow

13.Data Flow Explanation

1. Metrics Collection

Input:

Real-time operational data generated from the Kubernetes environment, including CPU usage, memory usage, network traffic, request rate, and active pod count.

Example:

CPU usage = 65%, Memory usage = 70%, Request rate = 120 requests/second, Active pods = 4.

Output:

Collected system metrics stored as monitoring data (time-series metrics) for further processing.

2. Data Preprocessing

Input:

Raw monitoring data collected from the Kubernetes environment.

Example:
CPU data: 65%, 67%, null Missing value corrected and normalized.
Output:
Cleaned and formatted data with removed noise, handled missing values, and normalized metrics ready for AI model input.

3. Workload Prediction
Input:
Preprocessed real-time metrics and historical workload data.
Example:
Historical data shows that request rates increase daily between **9:00 AM – 11:00 AM**.
Output:
Predicted future workload demand such as expected CPU usage, memory usage, request rate, and estimated resource requirements.
Example Output:
Predicted CPU usage = 85%
Predicted request rate = 200 requests/second

4. Carbon Optimization
Input:
Predicted workload demand and possible scaling scenarios.
Example:
Option 1: Add 3 pods → Higher energy consumption
Option 2: Add 2 pods → Lower energy consumption
Output:
Estimated energy consumption and carbon emission values for different scaling options.
Example Output:
Option 1: 0.9 kg CO_2
Option 2: 0.6 kg CO_2

5. Job Priority Scheduling
Input:
Incoming job requests including job type, execution time, and priority level.
Example:
Payment processing request → High priority
System log backup → Low priority
Output:
Classified jobs with scheduling decisions.
Example Output:
Payment processing → Execute immediately
Log backup → Delay execution by 10 minutes

6. Scaling Decision
Input:
Workload prediction results, carbon optimization analysis, and job priority scheduling results.
Example:
Predicted CPU = 85%
Carbon optimization suggests using 2 pods instead of 3.

Output:
Final scaling action decision.
Example Output:
Scale up from **4 pods → 6 pods**.

7. Reporting and Dashboard
Input:
System performance data, scaling decisions, energy consumption results, and job execution details.
Example:
CPU usage after scaling = 60%
Energy consumption reduced by 20%.
Output:
Visual dashboards and reports showing system performance, resource usage, carbon emissions, and workload management results.

12.5 Scalability and Security Considerations

1. Scalability Considerations

Scalability is an essential requirement for modern cloud-based DevOps systems because workloads can change dynamically depending on user demand. Cloud-native infrastructures often rely on container orchestration platforms such as Kubernetes to manage and scale applications efficiently. Kubernetes allows applications to run as containerized microservices and supports automatic scaling of pods and resources when system demand increases. This capability enables systems to maintain performance and reliability even under heavy workloads while ensuring efficient resource utilization [15].

In addition, microservice-based DevOps architectures provide modular scalability. Each service can be deployed and scaled independently without affecting other system components. This approach improves system flexibility and allows organizations to expand infrastructure gradually as the workload grows. Such scalable DevOps frameworks help organizations maintain high availability, fault tolerance, and operational efficiency in cloud-native environments [15].

Another important aspect of scalability is the ability to optimize resource allocation to reduce energy consumption and environmental impact. Recent research highlights that sustainable DevOps practices should incorporate intelligent resource management and predictive optimization techniques to improve infrastructure efficiency. By analyzing workload patterns and adjusting system resources dynamically, organizations can reduce unnecessary infrastructure usage while maintaining system performance [13].

Furthermore, modern DevOps pipelines generate large volumes of operational data during system monitoring and deployment processes. Effective monitoring and automation tools help manage these large data streams and enable efficient system scaling decisions. Continuous monitoring and feedback loops between development and operations teams improve system responsiveness and allow faster detection of performance issues in large-scale environments [16].

2. Security Considerations

Security is a critical concern in DevOps environments because software systems are continuously deployed and updated through automated pipelines. Continuous delivery pipelines are considered critical assets that must be protected from potential security vulnerabilities and system failures. If these pipelines are compromised, attackers could gain access to production systems or manipulate deployment processes [13].

One important security practice is the implementation of strong access control mechanisms. Role-based access control (RBAC) can be used to ensure that only authorized users and services have permission to modify infrastructure resources or access sensitive operational data. Proper authentication and authorization mechanisms help prevent unauthorized system access and reduce potential security risks.

Another key security measure involves securing communication between different components of the DevOps infrastructure. Cloud-based systems typically consist of multiple interconnected services, APIs, and monitoring tools. These components must exchange data securely using encrypted communication protocols to prevent interception or unauthorized modification of operational data.

Monitoring and logging systems also play an important role in maintaining security. DevOps platforms generate large amounts of operational data, which can be analyzed to detect anomalies, system failures, or potential cyber-attacks. Research shows that automated monitoring systems can improve operational awareness and provide early alerts when abnormal behavior is detected within software systems [6].

Finally, integrating security practices directly into the DevOps lifecycle—often referred to as DevSecOps—helps ensure that security is continuously enforced throughout the software development and deployment process. By embedding security testing, vulnerability scanning, and automated compliance checks into CI/CD pipelines, organizations can reduce risks and maintain secure software delivery environments [13].

14.USER REQUIREMENTS

13.1 Stakeholder Identification

1. **Cloud Infrastructure Administrators:** These users monitor resource usage and manage pods to ensure the system operates efficiently. They require clear visibility into system performance, pod scaling, and energy consumption.
2. **DevOps Engineers:** DevOps engineers deploy and maintain applications on Kubernetes clusters. They require automated scaling and priority scheduling to reduce manual workload and maintain service stability.
3. **Application Users / End-Users:** These are the people using services hosted on the cloud, such as web or e-commerce applications. They expect high availability, fast response times, and minimal service interruptions.
4. **Business / Operations Managers:** Managers are interested in reducing operational costs and meeting sustainability targets. They require reports on energy consumption, carbon emissions, and overall system efficiency.

5. **Researchers / System Developers:** These users develop and improve AI prediction models, carbon optimization algorithms, and scheduling mechanisms. They require access to system metrics, logs, and datasets to evaluate and enhance the framework.

13.2 Functional Requirements

1. The system must collect real-time metrics from Kubernetes clusters, including CPU, memory, network traffic, and active pods. These metrics are essential for AI prediction, carbon optimization, and scheduling decisions.
2. It must predict future workloads using LSTM or similar AI models. Predictions should be proactive and accurate to enable effective resource scaling.
3. The system must calculate energy usage and carbon emissions for scaling actions and make decisions that minimize resource consumption while maintaining performance.
4. It should schedule jobs based on priority, ensuring high-priority tasks run immediately and low-priority jobs may be delayed or merged to save energy.
5. The system must scale pods optimally using workload predictions and carbon-aware decisions to maintain availability and reduce idle resources.
6. The system must send alerts if high-priority jobs may be delayed or if resource usage exceeds limits.
7. Finally, it should integrate with existing DevOps tools like Kubernetes, Prometheus, and Grafana to use current monitoring and orchestration infrastructure effectively.

13.3 Non-Functional Requirements

1. **Security:** Monitoring data and logs must be protected from unauthorized access. Only authorized users or AI modules can control scaling or access sensitive metrics.
2. **Performance:** AI predictions must be delivered within a few seconds to allow proactive scaling. Scaling and scheduling decisions should not delay services.
3. **Usability:** Dashboards and reports must be intuitive and easy for administrators and managers to interpret.
4. **Reliability:** The system must remain highly available and fault-tolerant. Scaling and scheduling must not interrupt running services.
5. **Scalability:** The system must support small to large Kubernetes clusters and handle varying workloads without performance degradation.
6. **Maintainability:** AI models, scheduling algorithms, and monitoring configurations should be easy to update.
7. **Sustainability:** The system must continuously optimize energy usage and reduce carbon emissions while maintaining performance.

8. **Compatibility:** It must work with existing cloud infrastructure, container orchestration, and DevOps tools like Kubernetes, Prometheus, Grafana, and Docker.

15.COMMERCIALIZATION PLAN

14.1 Target Market and Customer Persona

The proposed AI-Driven Green DevOps Optimization System targets organizations that rely heavily on cloud infrastructure and Kubernetes-based deployments. These organizations aim to reduce operational cost, improve resource utilization, and minimize carbon emissions from cloud operations.

Target Market

The main markets include:

- Cloud-native companies running microservices on Kubernetes
- Large enterprises using DevOps pipelines
- Software development companies
- FinTech and e-commerce platforms
- Cloud service providers
- Organizations focusing on Green IT and sustainability

These organizations face challenges such as inefficient auto-scaling, energy waste in idle workloads, and lack of carbon-aware infrastructure management.

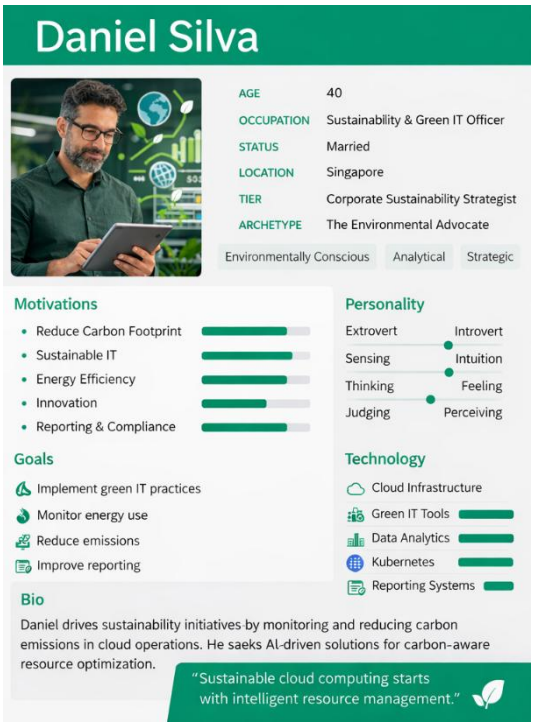


Figure 4

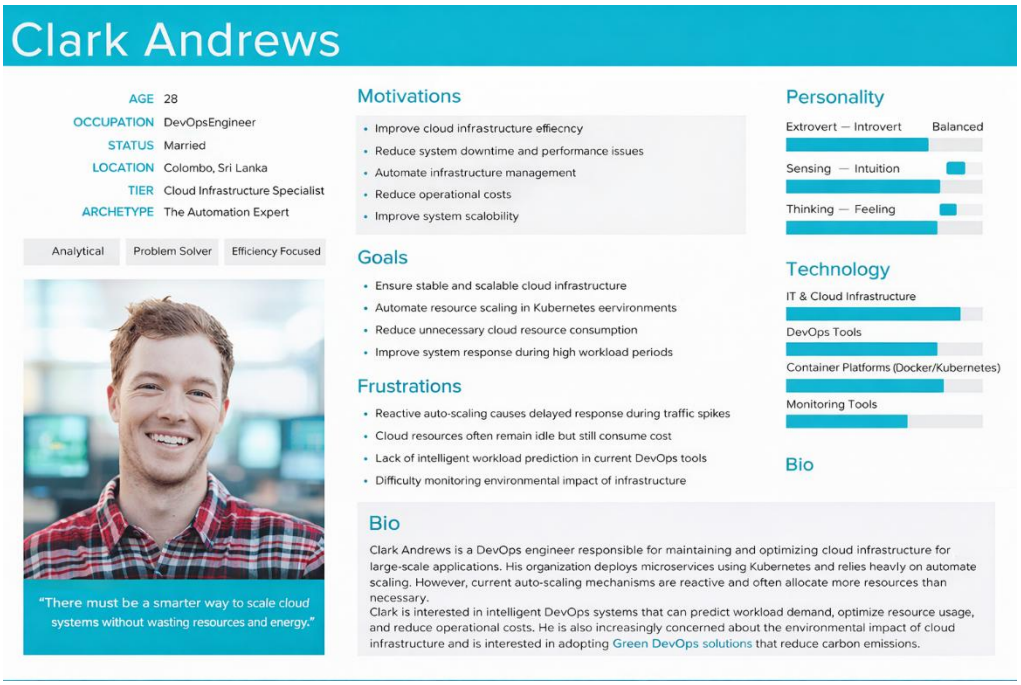


Figure 5

14.2 Value Proposition

The proposed system provides a **smart DevOps optimization platform** that enhances both system performance and environmental sustainability.

The key value offered includes:

1. **Predictive Scaling** – AI models predict future workloads and enable proactive resource allocation.
2. **Energy Efficiency** – Carbon-aware optimization reduces unnecessary energy usage in cloud environments.
3. **Operational Cost Reduction** – Efficient resource allocation lowers cloud infrastructure costs.
4. **Improved Workload Management** – Priority-based scheduling ensures critical tasks receive resources first.
5. **Sustainable Cloud Operations** – Helps organizations reduce their carbon footprint and support green computing initiatives.

This combination provides organizations with **better performance, lower operational costs, and environmentally responsible cloud management**.

14.3 Revenue Model

Several revenue models can be used to commercialize the system:

1. **Software-as-a-Service (SaaS)**
   Organizations subscribe to the platform and integrate it with their DevOps environments.
2. **Enterprise Licensing**
   Large organizations can purchase licenses to deploy the system within their private infrastructure.
3. **Usage-Based Pricing**
   Charges are based on the number of nodes, workloads, or Kubernetes clusters managed by the system.
4. **Consulting and Integration Services**
   Companies may pay for professional services to integrate the platform into existing DevOps pipelines.

14.4 Cost Estimation

The estimated costs required to develop and commercialize the proposed system are as follows:

Cost Category	Description	Estimated Cost (LKR)
Development Cost	Software development, AI model design, system architecture implementation	1,200,000
Infrastructure Cost	Cloud servers, Kubernetes clusters, monitoring tools, and storage	500,000
Maintenance Cost	System updates, bug fixes, and performance improvements	300,000
Customer Support	Technical support, onboarding, and user training	200,000
Marketing and Promotion	Advertising, product demonstrations, and partnerships	250,000

Table 2 : Cost Estimation

Estimated Total Cost: LKR 2,450,000

These costs represent the resources required to build, deploy, maintain, and promote the system in a commercial environment.

14.5 Pricing Strategy

The pricing strategy is designed to support organizations of different sizes while ensuring sustainable revenue generation.

Possible pricing tiers include:

Starter Plan
Suitable for startups and small development teams with limited workloads.

Professional Plan
Designed for medium-sized organizations that operate multiple Kubernetes clusters.

Enterprise Plan
Custom pricing for large enterprises requiring advanced features, integration support, and dedicated services.

Pricing may depend on factors such as:

- Number of monitored nodes
- Number of workloads managed
- Kubernetes cluster size
- Advanced analytics features

14.6 Competitive Advantage

The proposed system provides several advantages over traditional DevOps tools:

1. **AI-Based Predictive Scaling**
   Unlike conventional autoscaling systems that react to workload spikes, this system predicts demand in advance.
2. **Carbon-Aware Optimization**
   Most DevOps tools focus only on performance, whereas this framework also minimizes carbon emissions.
3. **Integrated Workload Prioritization**
   Combines AI prediction, carbon optimization, and job scheduling in one unified system.
4. **Cost Efficiency**
   Reduces unnecessary resource usage and lowers cloud infrastructure costs.

These capabilities provide a **unique competitive advantage in sustainable DevOps and green cloud computing**.

14.7 Intellectual Property Considerations

To protect the innovation and ensure commercial value, the following intellectual property strategies can be considered:

1. **Software Copyright**
   The source code and architecture of the framework can be protected through copyright laws.
2. **Patent Opportunities**
   Novel methods for AI-based predictive scaling and carbon-aware optimization may qualify for patent protection.

3. **Trade Secrets**
   AI models, scheduling algorithms, and optimization logic may be maintained as confidential intellectual property.

4. **Open-Source Integration**
   While the framework uses open-source technologies such as Kubernetes and Prometheus, the unique AI optimization framework remains proprietary.

Protecting intellectual property ensures the system maintains **commercial value, market advantage, and long-term sustainability**.

16.BUDGET AND JUSTIFICATION

Category	Item	Estimated Cost (LKR)	Justification
Hardware Costs	Raspberry Pi (for monitoring prototype)	30,000	Raspberry Pi devices can be used to simulate edge monitoring nodes that collect temperature and energy-related metrics from servers or experimental setups.
	Heat / Temperature Sensors	10,000	Sensors help collect environmental data such as temperature around computing devices. This data can be used to analyze hardware energy behavior and improve carbon optimization decisions.
	External Storage (1TB SSD / HDD)	25,000	Used to store system logs, datasets, experiment results, and backups generated during research.
Software Licenses	-	0.0	-
Cloud Services	Oracle Cloud Infrastructure (OCI) Usage	15,000	Oracle Cloud will be used to deploy Kubernetes clusters, run workload simulations, and test predictive scaling and carbon optimization algorithms in a realistic cloud environment.
Human Effort Cost	Research and Development	200,000	Covers effort required for system design, AI model development, framework implementation, and system integration.
Total		**280,000**	

Table 3: Budget and Justification

17.WORK BREAKDOWN STRUCTURE (WBS)

WBS ID	Task Description	Timeline
1	Research Planning	Dec 2025 – Jan 2026
1.1	Topic Selection and Idea Discussion	Dec 2025
1.2	Literature Review	Dec 2025 – Jan 2026
1.3	Research Gap Identification	Jan 2026
2	Research Design	Jan 2026 – Mar 2026
2.1	System Architecture Design	Feb 2026
2.2	Research Proposal Writing	Feb 2026
2.3	Proposal Submission and Presentation	Mar 2026
3	Data Preparation	Mar 2026 – Apr 2026
3.1	Dataset Collection	Mar 2026 – Apr 2026
4	AI Model Development	Apr 2026 – Jul 2026
4.1	AI Workload Prediction Engine Development	Apr 2026 – May 2026
4.2	Progress Presentation 1	May 2026
4.3	AI Model Training and Improvement	Jun 2026 – Jul 2026
4.4	Carbon Emission Optimization Engine Development	Jun 2026 – Aug 2026
4.5	Job Prioritization and Pod Scaling Logic	Jul 2026 – Aug 2026
4.6	AI Prediction Model Testing	Jul 2026
5	System Integration	Aug 2026
5.1	Integration of AI Prediction, Carbon Optimization and Scheduling Components	Aug 2026
6	System Evaluation	Aug 2026 – Sep 2026

6.1	AI Model Performance Analysis	Aug 2026
6.2	Carbon Emission Optimization Testing	Aug 2026
6.3	System Testing and Performance Evaluation	Sep 2026
6.4	Progress Presentation 2	Sep 2026
7	Documentation and Finalization	Sep 2026 – Oct 2026
7.1	Final Report and Project Website Development	Sep 2026 – Oct 2026
7.2	Research Paper Preparation	Oct 2026
8	Project Completion	Oct 2026 – Dec 2026
8.1	Final Presentation and Viva	Oct 2026
8.2	Research Paper Submission	Nov 2026
8.3	Publication Evidence Submission	Dec 2026

18.GANTT CHART

	Semester 1							Semester 2					
Task	**Dec 2025**	**Jan 2026**	**Feb 2026**	**Mar 2026**	**Apr 2026**	**May 2026**	**Jun 2026**	**Jul 2026**	**Aug 2026**	**Sep 2026**	**Oct 2026**	**Nov 2026**	**Dec 2026**
Topic Selection & Idea Discussion	■												
Literature Review	■	■											
Research Gap Identification		■											
System Architecture Design		■	■										
Research Proposal Writing			■	■									
Proposal Submission & Presentation				■									
Dataset Collection				■	■								
AI Workload Prediction Engine Development					■	■							
Progress Presentation 1						■							
AI Model Training & Improvement							■	■					
Carbon Emission Optimization Engine							■	■	■				
Job Prioritization & Pod Scaling Logic								■	■				
AI Prediction model Testing								■					
System Integration									■	■			
Progress Presentation 2									■				
System Testing & Evaluation										■	■		
Final Report & Website										■	■		
Final Presentation & Viva											■		
Research Paper Submission											■		
Publication Evidence Submission													■

REFERENCES LIST

[1] P. Xu, X. Li, and

Y. Zhang, “Predictive Auto-scaling in Cloud Computing Using LSTM,” *IEEE Transactions on Cloud Computing*, 2022.

[2] A. Li, J. Wu, and H. Zhao, “Carbon-Aware Cloud Resource Management,” *Journal of Sustainable Computing*, 2021.

[3] S. Chen and L. Wang, “Priority-Aware Job Scheduling in Cloud Systems,” *Journal of Cloud Computing*, vol. 11, p. 3, 2022.

[4] R. Patel, K. Heller, and R. Harris, “Limitations of Reactive Autoscaling in Kubernetes,” *ACM Computing Surveys*, 2023.

[5] J. Baliga, R. Ayre, K. Hinton, and R. S. Tucker, “Green Cloud Computing: Balancing Energy in Processing, Storage, and Transport,” *Proceedings of the IEEE*, vol. 99, no. 1, pp. 149–167, 2011.

[6] M. Zaharia et al., “Delay Scheduling: A Simple Technique for Achieving Locality and Fairness in Cluster Scheduling,” *Proceedings of EuroSys*, 2010.

[7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[8] H. Allam, *Sustainable Cloud Engineering: Optimizing Resources for Green DevOps*, 2023.

[9] L. P. Rongali, Green DevOps Metrics for Utility Operations, 2025.

[10] T. F. Düllmann et al., Exploiting DevOps Practices for Dependable and Secure Continuous Delivery Pipelines, 2018.

[11] N. Kumar and N. Sharma, Green Computing-Based DevOps Process in Agile Delivery Model, 2022.

[12] N. R. Pinnapareddy, Carbon Conscious Scheduling in Kubernetes to Cut Energy Use and Emissions, 2025.

[13] H. Chamarthi and M. Muppala, *AI Integration for Sustainable DevOps & Cloud: Framework for Optimized Resource Management and Carbon Efficiency*, 2024.

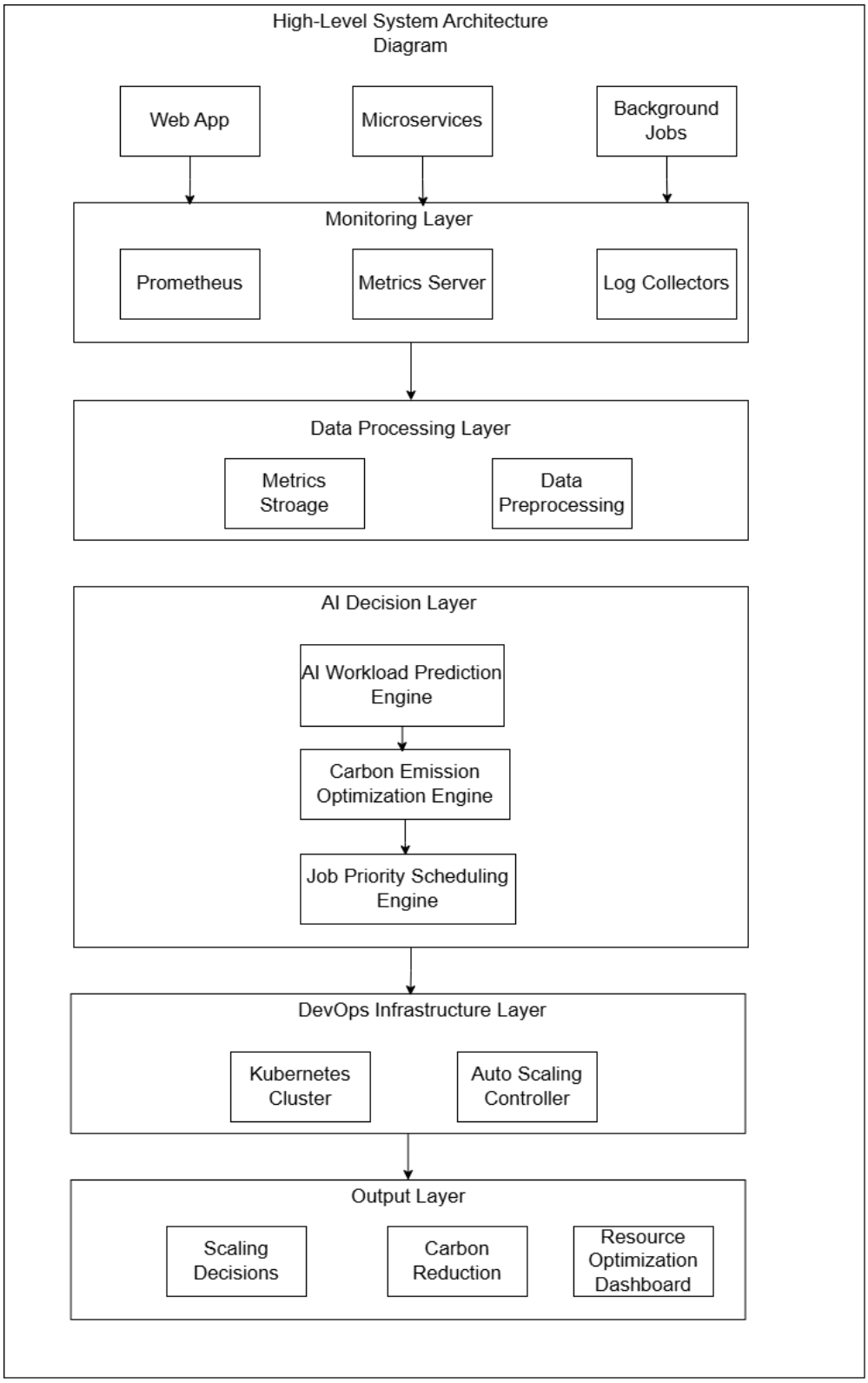
[14] T. F. Düllmann, C. Paule, and A. van Hoorn, “Exploiting DevOps Practices for Dependable and Secure Continuous Delivery Pipelines,” 2018.

[15] S. S. Poojary, S. Kamble, and G. S. Mamatha, “A Scalable DevOps Framework Using Kubernetes-Orchestrated Microservices for Cloud-Native Infrastructure.”

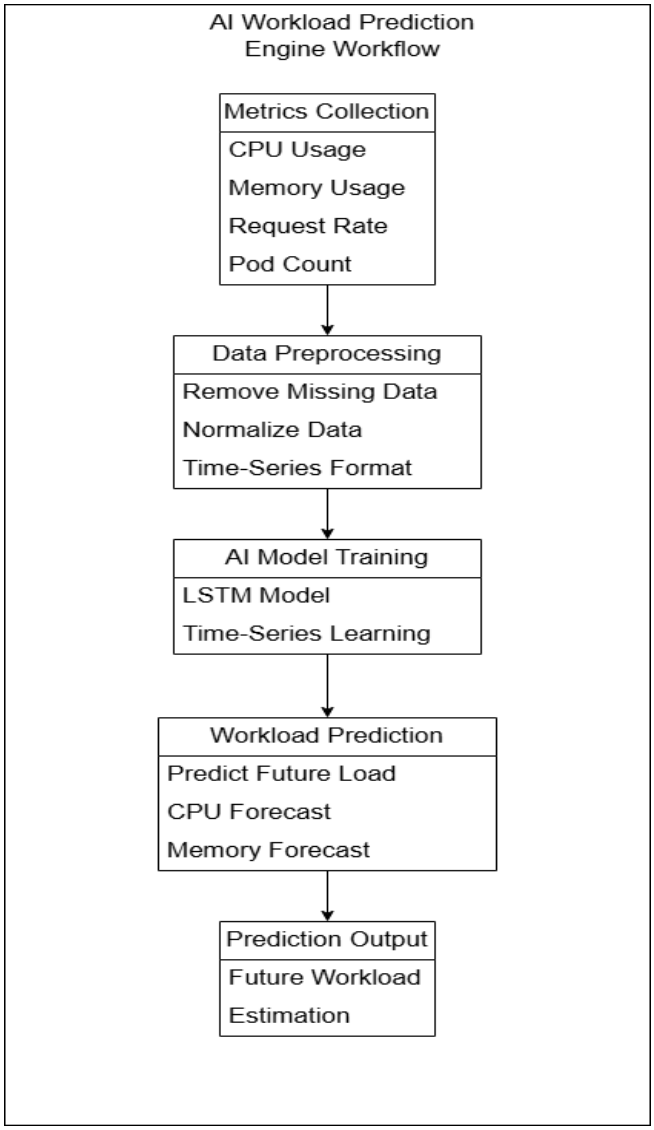
[16] A. Hrusto, P. Runeson, and E. Engström, “Closing the Feedback Loop in DevOps Through Autonomous Monitors in Operations,” 2021.

19.APPENDICES

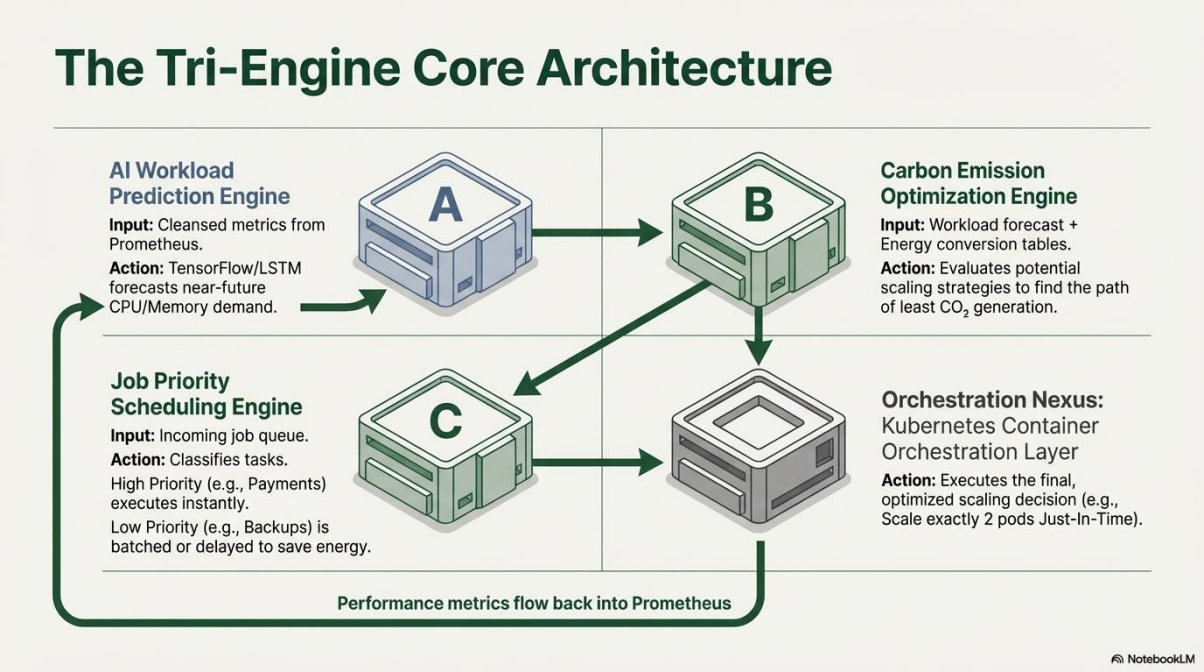
19.1 Detailed Diagrams



Appendix 1: High-Level System Architecture Diagram



Appendix 2: Carbon Optimization Decision Process



Appendix 3: Overview Diagram

19.2 Risk Analysis

Several risks may arise during the development and deployment of the proposed system.

Risk	Description	Mitigation Strategy
Data Availability	Limited historical monitoring data for training AI models	Use public datasets and simulated workloads for initial model training
Prediction Accuracy	AI predictions may be inaccurate during early stages	Continuously retrain the model using updated monitoring data
System Integration	Difficulty integrating AI models with Kubernetes infrastructure	Use standard APIs and DevOps tools for integration
Resource Constraints	Limited cloud resources for experimentation	Optimize experiments and use scalable cloud environments

Appendix 2: Risk Analysis